194.048 Data-intensive Computing 2026S

Entry Exercise

March 2026

Introduction

This exercise is required to qualify for a grade in this lecture.

Goal

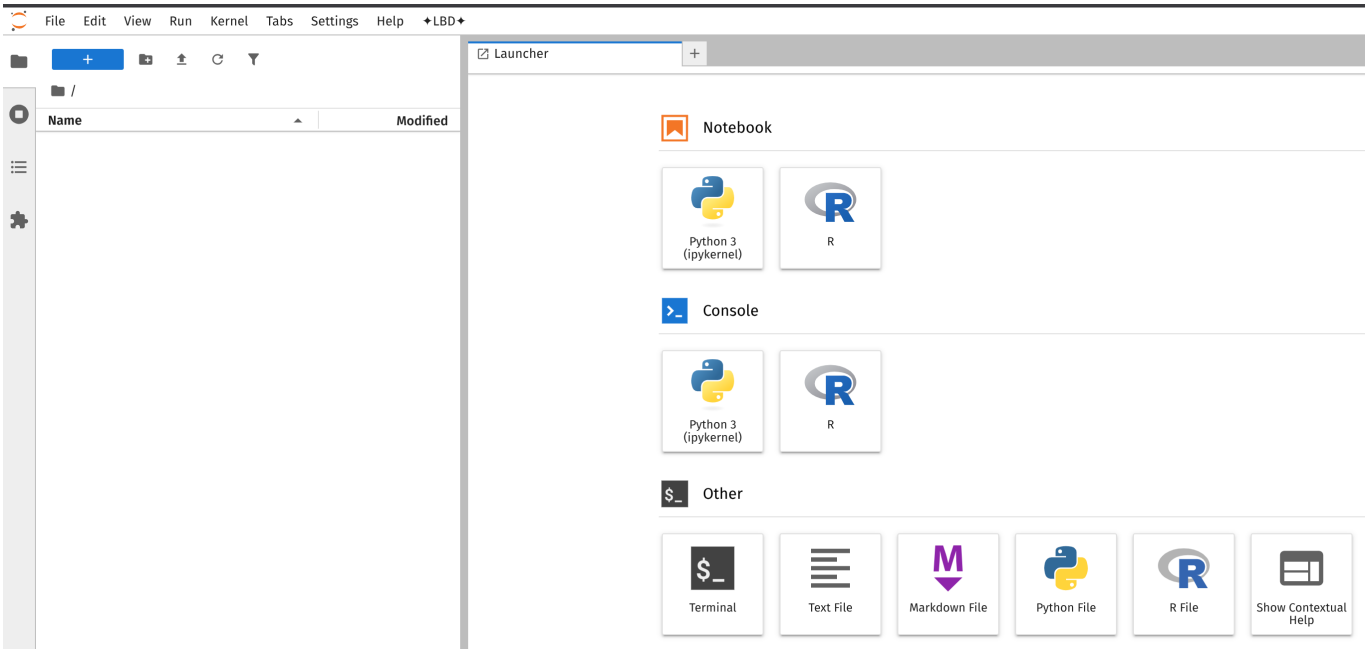
This exercise has three goals:

- Familiarize yourself with the Hadoop Distributed File System (HDFS) and learn how to interact with it.
- Run your first MapReduce job to count the words on a set of documents.

Cluster Access and Data

To access the LBD Cluster Environment, visit https://lbd.tuwien.ac.at
Note: TU VPN connection required when accessing from an external network!

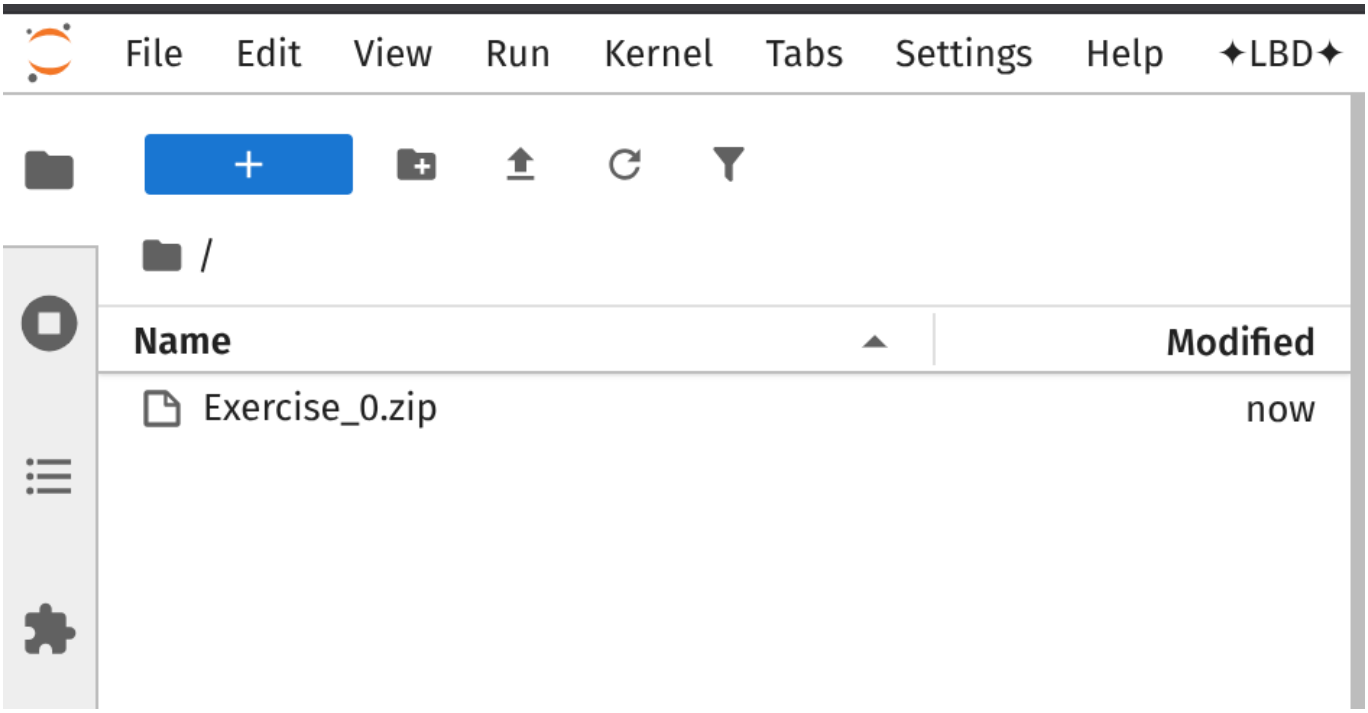
The environment looks something like this:



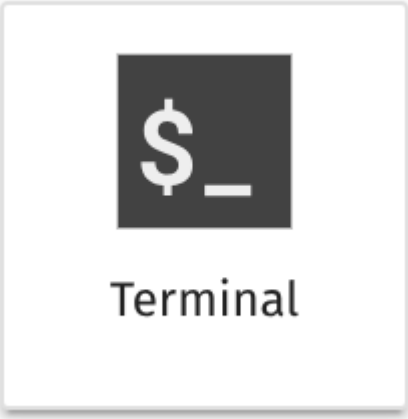
On the left side, you can see a file browser window where you can manage your assignment files, and in the middle to the right, you can see the launcher where you can start various applications, e.g., a new terminal to execute shell commands.

Preparing your Data

Download *Exercise 0.zip* from **TUWEL** and copy the zip file from your local directory to your home directory on the cluster (e.g., drag-and-drop to your LBD file browser). You can see an example below:



Then, in your launcher, start a new Terminal



and run

```
unzip Exercise_0.zip
```

Change into the created directory, and listing contents

```
cd Exercise_0
ls -1
```

should result in an output like:

```
SparkTest
WordCount
gutenberg_books
```

HDFS: the Hadoop Distributed File System

To familiarize yourself with HDFS, you will use a few E-Books downloaded from Project Gutenberg and store them in the distributed file system for later analysis. The books you will place on the HDFS are stored in the `gutenberg_books` subfolder, which contains 6 text files with works by Jonathan Swift:

```
ls -1 gutenberg_books
```

```
1080-0.txt
61179-0-it.txt
829-0.txt
pg1090.txt
pg14353-pm.txt
pg623.txt
```

Uploading (ingesting) files to HDFS

First, create a directory for you in HDFS:

```
hadoop fs -mkdir hdfs:///user/{yourUser}/books
```

To put a local file into an existing directory in HDFS (books), use the command:

```
hadoop fs -put {localFile} hdfs:///user/{yourUser}/books/
```

To copy all the files for this exercise, use the following command:

```
hadoop fs -put ./gutenberg_books/* hdfs:///user/{yourUser}/books
```

Some more HDFS commands to play with:

Here are some additional HDFS commands to explore and interact with the distributed file system:

```
hadoop fs -ls hdfs:///user/{yourUser}/
```

This command lists the contents of your user directory in HDFS.

```
hadoop fs -du -h hdfs:///user/{yourUser}/books
```

This command displays the disk usage of the `books` directory in a human-readable format.

```
hadoop fs -tail hdfs:///user/{yourUser}/books/pg1090.txt
```

This command shows the last part of the file `pg1090.txt`.

```
hadoop fs -cat hdfs:///user/{yourUser}/books/pg1090.txt | head
```

This command displays the first few lines of the file `pg1090.txt`.

MapReduce - Word Count

The original MapReduce paper used word count over a set of documents as an example. This example has become the “Hello World!” of MapReduce. Structurally, it is similar to building an index and other web analytics problems.

You can find the source code in the `WordCount/src` subfolder. The code consists of a job file `wordcount.py` and a runner file `DIC_runner.py`. In most applications, including this one, the job file alone usually suffices. However, a runner is useful when you want to do something with the output, such as writing it to a file.

Submit the MapReduce Job:

To submit the MapReduce job, follow these steps (takes a few seconds):

```
cd ~/Exercise_0/WordCount
python DIC_runner.py --hadoop-streaming-jar \
/usr/lib/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-r hadoop hdfs:///user/{yourUser}/books > word_count.txt
```

This job calls the runner file to Hadoop, takes in your previously loaded books as input, and writes out a text file with the results.

Export for Submission

After collecting the results of the MapReduce job, pick one word from the following list and check its frequency. Pick the word that corresponds to your birthday (e.g., if your birthday is June 1st, pick 1st word `always`, if it's January 31st, pick the 31st word `city`):

Word List:

always, music, those, both, mark, often, letter, until, mile, river, car, feet, care, second, call, first, side, work, place, live, small, large, spell, high, follow, such, stand, head, page, country, city

For the word you selected, check its frequency using the following command (replace `word` with the selected word):

```
grep -w 'word' word_count.txt > ../{yourUser}_ex0_result.txt
```

Spark - Approximate π

We want to check if Spark is running and configured properly for your account. Therefore, we will run the example provided to approximate π statistically. You can find the PySpark code in the `Exercise_0/SparkTest` subfolder.

```
cd ~/Exercise_0/SparkTest
cat calc_pi.py
spark-submit calc_pi.py >> pi.txt
cat pi.txt | tail -n 1 >> ../{yourUser}_ex0_result.txt
```

Submission to TUWEL

Check the generated result file. It should comprise a line with the word selected from the list and its frequency, and a line with a number close to π.

```
cd ..
cat {yourUser}_ex0_result.txt
```

You can download the file from the LBD file browser on your local machine and upload it to TUWEL.

IMPORTANT: Ensure the file is correctly formatted and contains the required information before submission.

Final Notes

- Make sure to follow all steps carefully to avoid errors.
- If you encounter any issues, consult the course documentation or contact the tutors.
- Double-check your results before submitting to TUWEL.

Additional Resources

For further reading and reference, consider the following resources:

- Hadoop Documentation: https://hadoop.apache.org/docs/

- Spark Documentation: https://spark.apache.org/docs/